

RELATORIO TECNICO

MoV UFU

Roteamento acessível no Campus Santa Mônica com OpenStreetMap

Modelagem do grafo, algoritmo de menor caminho e integração com reconhecimento de voz

Uberlândia - MG | 01/07/2026

Resumo executivo

Este relatório apresenta a reformulação do MoV UFU como uma aplicação de navegação pedestre baseada exclusivamente em dados do OpenStreetMap (OSM). A planta DWG/PDF deixa de compor a fonte operacional do sistema. Os caminhos, travessias e destinos são obtidos da base colaborativa OSM, convertidos para GeoJSON e transformados em um grafo ponderado sobre o qual é executado o algoritmo de Dijkstra.

A versão analisada contém 117 caminhos de pedestres, 32 pontos de travessia ou acessibilidade e 98 destinos nomeados. A rede possui 382 vértices, 398 arestas e seis componentes conectados. Para impedir a apresentação de rotas inexistentes, a aplicação opera sobre o maior componente, com 356 vértices e 81 destinos conectados.

Resultado principal: A aplicação passou a calcular rotas diretamente sobre a geometria OpenStreetMap. Escadas e trechos explicitamente marcados como wheelchair=no recebem penalidade elevada, enquanto trechos limitados ou sem pavimentação recebem custo adicional.

1. Concepção do projeto

O MoV UFU foi concebido para apoiar o deslocamento de pedestres no Campus Santa Mônica da Universidade Federal de Uberlândia, com ênfase em pessoas com mobilidade reduzida. A proposta combina cartografia colaborativa, teoria dos grafos, processamento geoespacial e uma interface web responsiva.

A reformulação substitui a dependência do desenho técnico por uma fonte geográfica aberta e continuamente atualizável. Essa decisão permite que correções feitas no OpenStreetMap sejam incorporadas ao sistema sem reeditar ou reexportar um arquivo CAD.

Nota de nomenclatura: Embora o diretório do projeto mencione o Problema do Carteiro Viajante, a função implementada é o menor caminho entre uma origem e um destino. Não se trata do Problema do Caixeiro Viajante nem do Carteiro Chinês, pois não há exigência de visitar todos os pontos ou percorrer todas as arestas.

2. Objetivos

1. Representar os caminhos de pedestres do campus como um grafo georreferenciado.
2. Calcular o menor percurso entre blocos e equipamentos nomeados no OSM.
3. Evitar ou desestimular escadas e trechos com restrições conhecidas de acessibilidade.
4. Permitir atualização reprodutível a partir da base pública do OpenStreetMap.
5. Disponibilizar seleção por listas, texto e comandos de voz em português brasileiro.

3. Fonte de dados e delimitação

A fonte primária é o OpenStreetMap, projeto colaborativo de dados geográficos abertos. A coleta abrange o limite cadastrado da Universidade Federal de Uberlândia no Campus Santa Mônica. O importador consulta a API OSM, identifica o polígono universitário e conserva apenas elementos localizados dentro desse limite.

Os dados são armazenados em GeoJSON com três classes: `pedestrian_path` para caminhos lineares, `accessibility_point` para travessias e elementos de acessibilidade, e `destination` para blocos, bibliotecas, equipamentos e outros locais nomeados.

4. Convenções de mapeamento no OpenStreetMap

Elemento	Tag principal	Uso no sistema
Caminho pedestre	<code>highway=footway/path/pedestrian</code>	Forma as arestas navegáveis.
Travessia	<code>highway=crossing</code>	Ponto de verificação de rampa e segurança.
Meio-fio rebaixado	<code>kerb=lowered</code> ou <code>kerb=flush</code>	Evidência de transição potencialmente acessível.
Escada	<code>highway=steps</code>	Recebe penalidade elevada.
Acesso por cadeira de rodas	<code>wheelchair=yes/limited/no</code>	Modifica o custo do trecho.
Piso tátil	<code>tactile_paving=yes/no</code>	Informação para avaliação de acessibilidade sensorial.

Elemento	Tag principal	Uso no sistema
Características físicas	surface, width, incline, smoothness	Base para evoluir o modelo de custo.

5. Processo de aquisição e preparação

6. Consultar a API pública do OpenStreetMap na caixa geográfica do campus.
7. Localizar o polígono com amenity=university e nome Universidade Federal de Uberlândia.
8. Filtrar caminhos, travessias, atributos de acessibilidade e destinos contidos no campus.
9. Converter geometrias e atributos para uma FeatureCollection GeoJSON.
10. Construir o grafo, identificar componentes conectados e selecionar a maior rede operacional.
11. Atualizar a interface, o vocabulário de voz e o cache do aplicativo.

Rede pedestre OpenStreetMap - Campus Santa Monica



Figura 1 - Rede pedestre extraída do OpenStreetMap. Linhas verdes representam caminhos, laranja representa travessias e azul representa destinos nomeados.

6. Modelagem do grafo

Cada coordenada consecutiva de uma LineString é convertida em vértice. Coordenadas iguais, arredondadas a sete casas decimais, compartilham a mesma chave e estabelecem a conectividade entre vias. Cada par consecutivo forma uma aresta bidirecional.

O comprimento físico da aresta é calculado pela fórmula de Haversine, apropriada para distâncias sobre a superfície terrestre. Os destinos nomeados são associados ao vértice mais próximo. Antes do roteamento, uma busca em profundidade identifica os componentes da rede; apenas o maior componente é oferecido como rede operacional.

7. Algoritmo de roteamento

O menor caminho é calculado pelo algoritmo de Dijkstra. Para cada vértice, o algoritmo mantém o menor custo conhecido desde a origem, seleciona o vértice não visitado de menor custo e relaxa as arestas adjacentes até atingir o destino ou esgotar a componente.

O custo de uma aresta corresponde ao seu comprimento multiplicado por um fator de acessibilidade. Trechos comuns usam fator 1. Trechos com wheelchair=limited ou surface=unpaved usam fator 4. Escadas e trechos com wheelchair=no usam fator 1000. A distância exibida ao usuário permanece a distância física; a penalidade serve apenas para escolher a rota.

Cuidado metodológico: Os fatores 4 e 1000 são heurísticas de projeto, não parâmetros normativos. Eles devem ser calibrados com usuários, auditorias de campo e requisitos de acessibilidade antes de uso institucional.

8. Arquitetura da aplicação

Camada de dados: GeoJSON gerado pelo importador Python a partir do OpenStreetMap.

Camada cartográfica: Leaflet com mosaicos do OpenStreetMap e sobreposição vetorial dos caminhos.

Camada de grafo: JavaScript no navegador para vértices, arestas, componentes e Dijkstra.

Camada de interação: Seletores de origem/destino, formulário textual e Web Speech API em pt-BR.

Distribuição: Aplicação web progressiva com manifesto e service worker para recursos locais.

9. Resultados atuais

A tabela resume o estado da extração realizada em 1 de julho de 2026. Os números podem variar a cada nova edição da comunidade OSM.

Indicador	Resultado
Caminhos de pedestres	117
Travessias/pontos de acessibilidade	32
Destinos nomeados na fonte	98
Vertices do grafo	382
Arestas do grafo	398
Componentes conectados	6
Vertices no maior componente	356
Destinos conectados operacionais	81

A existência de seis componentes demonstra que alguns caminhos ainda não compartilham vértices, mesmo quando parecem próximos visualmente. O bloco 1E, por exemplo, foi identificado em uma componente pequena na extração analisada; por isso não deve ser conectado artificialmente ao bloco 3E. A correção adequada é completar a geometria no OpenStreetMap após verificação local.

10. Reconhecimento de voz

O reconhecimento usa a Web Speech API configurada para português brasileiro. O sistema considera até cinco alternativas de transcrição, resultados intermediários e um limite de 15 segundos. Nomes e referências dos destinos OSM compõem o vocabulário dinâmico.

O analisador normaliza acentos, pontuação, números falados e contrações como de, do e da. Frases como 'do bloco 1A para o bloco 3E' são decompostas em origem e destino, desde que ambos estejam conectados à rede operacional.

11. Validacao, riscos e limitações

- 12.O OpenStreetMap e colaborativo e pode conter omissões, geometrias desconectadas ou atributos desatualizados.
- 13.Uma travessia mapeada não comprova a existencia de rampa, meio-fio rebaixado ou piso tatil.
- 14.A associacao de um prédio ao caminho mais próximo não substitui o mapeamento da entrada correta.
- 15.Trechos sem tags de acessibilidade são tratados como neutros, e não como comprovadamente acessíveis.
- 16.O sistema deve ser validado em campo e com pessoas com deficiência antes de orientar deslocamentos reais.

12. Protocolo recomendado de atualização

- 17.Realizar levantamento em campo com GPS e registro fotográfico autorizado.
- 18.Editar caminhos e atributos no editor iD, StreetComplete ou JOSM.
- 19.Garantir que caminhos que se cruzam compartilhem um vértice quando houver passagem fisica.
- 20.Registrar kerb, wheelchair, tactile_paving, incline, surface e width somente quando confirmados.
- 21.Executar tools/import_osm_santa_monica.py para atualizar o GeoJSON.
- 22.Executar os testes e revisar visualmente as rotas antes da publicacao.

13. Conclusão

A migração para OpenStreetMap torna o MoV UFU geograficamente consistente, aberto a atualizações comunitárias e independente de reexportacoes CAD. A representação em grafo e o algoritmo de Dijkstra fornecem uma base tecnicamente adequada para rotas ponto a ponto. O principal trabalho futuro e melhorar a completude dos atributos de acessibilidade e a conectividade da rede por meio de levantamento de campo.

A versao atual deve ser entendida como protótipo de pesquisa e apoio ao mapeamento, não como certificação de rota acessível. Sua evolução depende da qualidade dos dados, da participação dos usuários e da validação institucional.

Referências

- 23.[OpenStreetMap - pagina principal e dados cartograficos](#)
- 24.[OpenStreetMap Wiki - Key:wheelchair](#)

25. [OpenStreetMap Wiki - Key:kerb](#)
26. [OpenStreetMap Wiki - Key:ramp](#)
27. [OpenStreetMap Wiki - Key:tactile_paving](#)
28. [OpenStreetMap Wiki - Overpass Turbo](#)
29. [Leaflet - biblioteca de mapas web](#)